

TP2 : Les séries et dataframes avec Pandas

1. Séries et DataFrames

Dans Pandas, il existe deux types de structures de données : les séries et les dataframes.

Series 1		Series 2		Series 3		DataFrame
Mango		Apple		Banana		Mango Apple Banana
0 4		0 5		0 2		0 4 5 2
1 5		1 4		1 3		1 5 4 3
2 6		2 3		2 5		2 6 3 5
3 3		3 0		3 2		3 3 0 2
4 1		4 2		4 7		4 1 2 7

2. Manipulation des séries

Une Série est un tableau unidimensionnel étiqueté capable de contenir des données de n'importe quel type (entier, chaîne, flottant, objets python, etc.). Les étiquettes des axes sont collectivement appelées index. On peut choisir nos propres indexes (autres que 0, 1, ..).

Travail à faire :

1. Créez une série d'entiers S1. (N'oubliez pas d'importer pandas)

```
S1 = pd.Series([1, 2, 3, 5, 7])
```

2. Créez une série de nombres flottants S2 en spécifiant dtype = float

3. Créez une série S3 avec un index :

```
S3 = pd.Series([1, 5, 3, 9], index = ['a', 'b', 'c', 'd'])
```

4. Créez la série S4 suivante à partir d'un dictionnaire. (Les clés du dictionnaire seront les index de S4 et les valeurs ses éléments)

	Superficie
Paris	105,4
Marseille	240,6
Lyon	47,87
Toulouse	118,3
Nice	71,92

5. Affichez la superficie de Lyon

6. Affichez la ville ayant la plus grande superficie

7. Affichez la moyenne des superficies

3. Manipulation des dataframes

Un dataframe est une structure de données permettant de stocker les données selon deux dimensions : lignes et colonnes.

(a) Création d'un DataFrame

Travail à faire :

1. Utilisez le code suivant pour créer un dataframe :

```
df = pd.DataFrame( [ [2, 3], [5, 6],[8, 9] ],  
                  index=[ 'cobra', 'viper','sidewinder' ],  
                  columns=[ 'max_speed', 'shield' ])
```

Visualisez df.

Une autre méthode plus simple consiste à passer par un dictionnaire :

2. Créez le dataframe df1 suivant (à partir d'un dictionnaire):

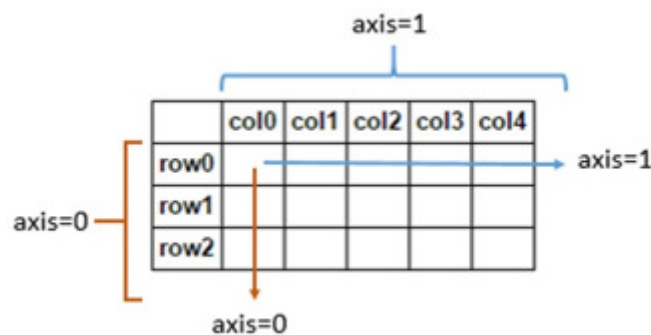
	Population	Superficie
Paris	2187526	105,4
Marseille	863310	240,6
Lyon	516092	47,87
Toulouse	479553	118,3
Nice	340017	71,92

2. Créez le dataframe df2 suivant :

	Brand	Price	Year
0	Honda	2200	2015
1	Toyota	2500	2013
2	Ford	2700	2018
3	Audi	3500	2018

(b) Accès aux éléments d'un DataFrame

Un dataframe se comporte comme un dictionnaire dont les clefs sont les noms des colonnes et les valeurs sont des séries. Les colonnes sont accessibles par leurs noms et les lignes par leurs index. Ce dernier peut-être un entier, une date, une chaîne de caractères.



Travail à faire : Dans cette partie, vous allez utiliser le dataframe df2

1. Affichez la colonne Brand
2. Affichez les 3 premières valeurs de la colonne Brand
3. Affichez les deux colonnes Brand et Price :

	Brand	Price
0	Honda	2200
1	Toyota	2500
2	Ford	2700
3	Audi	3500

4. Ecrivez la commande qui permet d'accéder à la valeur Honda

5. Ecrivez la commande qui permet d'accéder à la valeur 3500

6. Affichez seulement les deux premières lignes de df2

(c) Opérations sur les DataFrames : statistiques, filtrage, tri ..

Travail à faire :

1. Spécifiez le résultat des lignes suivantes :

```
df2.columns
```

```
df2.describe()
```

```
df2.corr()
```

2. Calculez les moyennes des colonnes (numériques)

3. Calculez la somme de chaque colonne

4. Filtrage :

On peut écrire des conditions sur les colonnes (cela reprend le principe des masques binaires sur les vecteurs numpy) : `df2[ "Price" ]>2500`

et se servir du résultat pour filtrer les lignes : `df2[ df2[ "Price" ]>2500 ]`

Remplacez les prix inférieurs à 2600 par 2000.

5. Pour trier un dataframe, on utilise `df2.sort_values()` et on spécifie la (ou les) colonne(s) en fonction de la(les)quelle(s) on va effectuer le tri. L'attribut `inplace = True` permet de mettre à jour df2 par la version triée.

Exemples à faire :

(a) `df2.sort_values(by=['Brand'], inplace=True, ascending=False)`

(b) Triez le dataframe df2 en fonction du prix (dans un ordre croissant)

(c) Triez le dataframe df2 en fonction de l'année et du prix (dans un ordre croissant)